

Assertion languages with modalities and separating connectives

Bartosz Bednarczyk
M1 student in Computer Science at ENS Paris-Saclay in 2017–2018
hosting institution: LSV, ENS Paris-Saclay
supervisor: Stephane Demri

Defense date: September 7th

The general context

The internship was about proving complexity bound of extensions of classical logics i.e. modal logics with constructs from separation logic. The question came from the recent paper by Demri and Fervari (AiML 2018) on modal separation logic.

The authors of the paper considered the logic when operator $\Diamond^{-1}$ and star $*$ are the only one allowed in the signature (among others like $\Diamond$ or universal modality) and they proved that the exact complexity lies between PSpace and Tower. The goal of the internship was to obtain the exact complexity of such logic.

The problem studied

The question we resolved is the exact complexity of the satisfiability problem for Computational Tree Logic over trees extended with propositional quantification, when only operator EX is allowed. It is important because it sharpens the previous Tower lower bound for the full logic studied by several people over last 30 years.

The proposed contribution

Majority of our proofs relies on a novel technique called local nominals, which imitates constructs from hybrid logics and could be interesting on its own. The hardness results depends on encoding of tiling systems and employing arithmetical relation on binary counters.

Arguments in favor of its validity

This is purely theoretical project and we believe that there is no notion of *stability* for it. Our proofs seems to work only for this specific problem.

Balance sheet and prospects

The next step is to work on the presentation side since proof are very technical and not easy to follow. Then, we would like to write the full version of the report and send it to LICS 2019 conference.

Assertion languages with modalities and separating connectives

Bartosz Bednarczyk

Laboratoire Spécification et Vérification, ENS Paris-Saclay, France

`bartosz.bednarczyk@ens-paris-saclay.fr`

Defense date: September 7th, 2018

Abstract

Quantified Computation Tree Logic QCTL extends the well-known temporal logic CTL with quantification over atomic proposition. While the logic was defined long time ago, only quite recently it was shown to have a non-elementary satisfiability problem under tree semantics by Laroussinie and Markey in LMCS 2014.

Motivated by this result, we tried to identify a natural fragment of QCTL enjoying an elementary complexity while allowing the full power of propositional quantification. In this paper we would like to describe how our research program failed. We will show that that the most natural fragment of QCTL, containing only **EX** modality, is already TOWER-hard. That sharpens the hardness result for the full logic. Additionally, we will turn our attention to a special case where tree models are of bounded degree and prove that the complexity drops to a class between NEXPTIME and EXPSpace.

1 Introduction

Temporal logics are clearly one of the most important formalisms in the area of program verification. Branching-time logics, e.g. the famous computational tree logic CTL [6], are especially useful expressing and automatically verifying (via model checking) properties of reactive systems [7].

However, the expressive power of CTL is very limited. For example, the logic cannot express even such simple property that a node in computation tree has an unique successor. To overcome this problem, several extensions of CTL with various forms of counting were proposed in the past (we recommend to read through a introduction section from [17] for a gentle survey).

In this report we focus on one of such extensions i.e. an extension of CTL with *propositional quantification*, originally introduced by Sistla [21], [10]. In that setting, propositional quantification allow us to add fresh atomic propositions in the model before evaluating the truth value of a temporal-logic formula. Obtained logic, referred therein as QCTL, has been extensively studied in [15, 12, 19, 13, 16, 8]. Most of algorithmic questions regarding QCTL, like model checking or satisfiability, have been solved under various semantic assumptions. Especially Laroussinie and Markey in [16] proved, that under tree semantics (when labellings refer to execution tree) the logic QCTL has (non-elementary) TOWER-complete complexity. We will revisit this result.

1.1 Our contribution

In this report we consider a fragment of QCTL interpreted under tree semantics when until operator **U** is disallowed from syntax. Such fragment (without quantification) is sometimes referred as UB (e.g. [23]), but we rather stay with a more meaningful name CTL_{EX} (and QCTL_{EX} for its extension with quantifiers). The main contributions presented in this paper are the following:

- We prove TOWER-hardness of satisfiability problem for QCTL_{EX}, which sharpens the complexity result for the full QCTL logic from [16].
- Then, we consider a satisfiability problem for QCTL_{EX} over trees with bounded degree. We prove NEXPTIME-hardness for the problem as well as membership to some subclass contained in EXPSpace called STA(*, 2^{O(n)}, n). We believe in STA(*, 2^{O(n)}, n)-completeness of the satisfiability problem in the bounded case. However we will omit the proof here.
- Finally, we provide some expressivity results regarding fragments of QCTL_{EX} with a single alternation.

Majority of our proofs relies on a novel technique called *local nominals*, which imitates constructs from hybrid logics and could be interesting on its own.

1.2 How QCTL_{EX} is connected with the original topic of the internship?

Modal separation logic MSL is a logic introduced recently by Demri and Fervari in [9], whose models are memory states as in separation logic and the logical connectives include modal operators as well as separating conjunction and magic wand. With these operators, some fragments of MSL can be seen as genuine modal logics whereas some others capture standard separation logics, leading to an original language to speak about memory states. The decidability status and the computational complexity of several fragments of MSL were established in [9].

During the internship we focused on a fragment of MSL, where only $\Diamond^{-1}$ (predecessor relation) and a star $*$ (separating conjunction) are the only operators allowed. Instead of giving the precise syntax and semantics, we rather describe how the logic works.

The recalled fragment of MSL can be seen as a standard modal logic over a functional frames (aka. heap or forest of rooted trees if we reverse the arrows), where each world is labeled with some unary predicates and an operator $\Diamond^{-1}$ is used to navigate through a heap. Additionally, one can use an operator star $*$. Such specific construct makes an evaluation of a formula dynamic, since star operator speaks about what happens when we split the heap into two disjoint pieces. It is not hard to see that such a fragment is decidable by employing a theorem about decidability of MSO with one functional symbol. However the exact complexity of the logic remains unknown and is somewhere between PSPACE and TOWER.

During my internship, we tried to solve the satisfiability problem for MSL directly. However a few failed attempts we decided to simplify the logic a little and think about models as trees rather than heaps (which not seems to be really a restriction since a given formula can move around heap with only a fixed number of steps). Then, we observed that a star operator $*$ can be simulated with propositional quantification $\exists p$ and, instead of splitting a model into two pieces, we can simply color the first part with p and the second one with $\neg p$. Finally in the case of trees $\Diamond^{-1}$ is the same as **EX** operator from CTL logic and that gave us the motivation to study QCTL logic instead.

Obtained logic QCTL_{EX} seems to be more powerful than MSL but also more elegant. We hoped for a very long time that existing techniques for QCTL would be able to help us and

obtain an elementary bound on the satisfiability problem for MSL. However we failed once more and we proved (which was very surprising for us!) that QCTL_{EX} is already TOWER-hard. Hence the exact complexity for the desired fragment for MSL is still open.

2 Preliminaries

2.1 Kripke structures and Tree unfoldings

Let AP be a fixed set of atomic propositions. We sometimes refer to its elements as *colors*. Below we present a classical definition of Kripke structure:

Definition 2.1. A Kripke structure $\mathcal{K}$ is a triple $(Q_{\mathcal{K}}, R_{\mathcal{K}}, l_{\mathcal{K}})$, where $Q_{\mathcal{K}}$ is a set of states (worlds), $R_{\mathcal{K}} \subseteq (Q_{\mathcal{K}})^2$ is a transition relation and $l_{\mathcal{K}} : Q_{\mathcal{K}} \rightarrow 2^{AP}$ is a labeling function.

In this report we will often work with rooted, directed and 2^{AP} -labeled trees.

Definition 2.2. A *tree* $\mathfrak{T} = (\mathcal{T}, r_{\mathcal{T}})$ is a pair consisting of a (possible infinite) Kripke structure $\mathcal{T} = (V_{\mathcal{T}}, E_{\mathcal{T}}, l_{\mathcal{T}})$ and a selected state $r_{\mathcal{T}} \in V_{\mathcal{T}}$ called *root*, where relation the $E_{\mathcal{T}}$ is acyclic and there are no in-going edges to the state $r_{\mathcal{T}}$.

In case of tress, we prefer to refer to set of states as *nodes* and we call their successors *children*. Additionally, we say that a tree is *d-bounded*, if every node has at most d out-going edges. A *binary* tree is a 2-bounded tree.

For a given Kripke structure $\mathcal{K}$ and a selected state $s \in Q_{\mathcal{K}}$ we define a 2^{AP} -labeled tree so-called $\mathfrak{T}_{\mathcal{K}, s_{\mathcal{T}}}$ tree unfolding (or tree unraveling). To construct such a tree, we start from a labeled tree $\mathfrak{T}_0$ containing a single of a single node $s_{\mathcal{T}}$ corresponding to a state s . Then (possibly infinitely many times) we pick an arbitrary node v_s corresponding to some state $s \in Q_{\mathcal{K}}$ and we add it fresh children in the way that they are labeled in the same way as successors of s in $\mathcal{K}$ (and in the way that their number is the same as in the original structure).

For the sake of accuracy we present the formal definition below:

Definition 2.3. A (possible infinite) tree $\mathfrak{T}_{\mathcal{K}, s} = (\mathcal{T}, s_{\mathcal{T}})$ is an unfolding of a Kripke structure $\mathcal{K} = (Q_{\mathcal{K}}, R_{\mathcal{K}}, l_{\mathcal{K}})$ from a selected state s if there is a mapping $h : V_{\mathcal{T}} \rightarrow Q_{\mathcal{K}}$ satisfying all the following properties:

- Function h is a homomorphism i.e. for any two nodes $v, v' \in V_{\mathcal{T}}$ satisfying $E_{\mathcal{T}}(v, v')$ we also have $R_{\mathcal{K}}(h(v), h(v'))$.
- Root $s_{\mathcal{T}}$ is mapped to the state s i.e. $h(s_{\mathcal{T}}) = s$.
- Mapping h do not violate labels i.e. $h(v) = s$ implies $l_{\mathcal{T}}(v) = l_{\mathcal{K}}(s)$.
- Degree coincide i.e. for an arbitrary node $v \in V_{\mathcal{T}}$ the number of children of v is equal to out-degree of a state $h(v) \in Q_{\mathcal{K}}$.

Since that definition is rather complicated we will often refer to an intuitive meaning of an unfolding.

2.2 Quantified Computational Tree Logic

We consider an extension of a standard CTL with quantification over atomic propositions. The syntax of QCTL is as follows:

$$\varphi ::= p \mid \neg\varphi \mid \varphi \wedge \varphi \mid \mathbf{EX}\varphi \mid \mathbf{AX}\varphi \mid \exists p \varphi \mid \mathbf{E}(\varphi \mathbf{U} \varphi) \mid \mathbf{A}(\varphi \mathbf{U} \varphi)$$

where $p \in AP$. Other operators like **EF**, **EG**, **AG** and many others can be abbreviated using until **U** and negation. Additionally we define $\forall p.\varphi$ as $\exists p.\neg\varphi$. By dropping the last two pieces from the syntax above we obtain the logic QCTL_{EX} . When propositional quantifiers are not allowed in QCTL_{EX} we refer to the logic as CTL_{EX} .

To denote QCTL_{EX} formulae we use Greek letters, usually φ or ψ , with subscripts. We define the length $|\varphi|$ of a given formula φ as a total number of bits required to write a formula. A *modal depth* $md(\varphi)$ of a formula φ is the deepest nesting of modal operators **EX**.

2.3 Tree semantics

Propositional quantifiers $\exists p.\varphi$ can be interpreted in various ways (see e.g. [12] or [16]). In this report we will consider so-called *tree semantics*, which corresponds to labeling an execution tree rather than states from Kripke structure. We stress here that such design makes a crucial difference is that the very same state from a Kripke structure can be labeled in many different ways in its tree unraveling.

To define a semantics properly, we first define p -enhancement of a Kripke structure $\mathcal{K}$. We can intuitively think about enhancement as adding a new letter to an alphabet AP .

Definition 2.4. Let p be a unary predicate. A p -enhancement $\mathcal{K}_{+p}$ of a Kripke structure $\mathcal{K}$ is a Kripke structure $\mathcal{K}' = (Q'_{\mathcal{K}}, R'_{\mathcal{K}}, l'_{\mathcal{K}})$ such that both structure coincides on sets of states $Q'_{\mathcal{K}} = Q_{\mathcal{K}}$ and transition functions $R'_{\mathcal{K}} = R_{\mathcal{K}}$ but a type of $l'_{\mathcal{K}}$ is extended to $l'_{\mathcal{K}} : 2^{AP \cup \{p\}} \rightarrow Q'_{\mathcal{K}}$ in such way that after removing p from $l'_{\mathcal{K}}$ two labellings $l'_{\mathcal{K}}$ and $l_{\mathcal{K}}$ coincide again i.e. for every $s \in Q_{\mathcal{K}}$ we have $l_{\mathcal{K}}(s) = l'_{\mathcal{K}}(s) \setminus \{p\}$.

We use $\mathfrak{T}_{+p}$ to denote a p -enhancement of a tree $\mathfrak{T}$ i.e. a p -enhancement of its underlying Kripke structure.

Let $\mathfrak{T}$ be an unfolding of some Kripke structure and let $v \in V_{\mathfrak{T}}$. With $\mathfrak{T}, v \models \varphi$ we denote that a formula φ is satisfied at a node v in tree $\mathfrak{T}$. We define it as follows:

$\mathfrak{T}, v \models p$	iff	$p \in l_{\mathfrak{T}}(v)$
$\mathfrak{T}, v \models \neg\varphi$	iff	$\mathfrak{T}, v \not\models \varphi$
$\mathfrak{T}, v \models \neg\varphi \wedge \psi$	iff	$\mathfrak{T}, v \models \varphi$ and $\mathfrak{T}, v \models \psi$
$\mathfrak{T}, v \models \mathbf{EX}\varphi$	iff	$\exists w \in V_{\mathfrak{T}} \text{ s.t. } \mathfrak{T}, w \models \varphi \text{ and } E_{\mathfrak{T}}(v, w)$
$\mathfrak{T}, v \models \mathbf{AX}\varphi$	iff	$\mathfrak{T}, v \models \neg\mathbf{EX}\neg\varphi$
$\mathfrak{T}, v \models \exists p \varphi$	iff	$\exists \mathfrak{T}_{+p} \text{ s.t. } \mathfrak{T}_{+p}, v \models \varphi$

Semantics for other operators can be easily obtained with above rules.

The *satisfiability problem* is the following: for a given formula φ from logic (i.e. QCTL or QCTL_{EX}) we ask whether there exists a rooted tree model s.t. φ holds at its root.

We recall the main result from [16] regarding the satisfiability problem for the full QCTL logic.

Theorem 2.1 ([16]). *The satisfiability problem for QCTL is TOWER-complete, even when trees are required to be binary.*

2.4 Complexity classes

We consider standard complexity classes like PSPACE, EXPTIME, EXPSPACE or k -EXPTIME (k -fold exponential time). More details about them are available in textbook [2]. The class of problems of time complexity bounded by a tower of exponentials, whose height is an elementary function of the input is called TOWER [20].

We assume that the reader is also familiar with alternating Turing machines. A famous characterization by Chandra, Kozen and Stockmeyer, states that EXPSPACE corresponds to class of alternating Turing machines working in exponential time with unbounded number of alternations [5].

A less-known yet useful complexity class is $\text{STA}(*, 2^{\mathcal{O}(n)}, n)$. It corresponds to alternating Turing machines working in exponential time with linear number of alternations [3]. The class $\text{STA}(*, 2^{\mathcal{O}(n)}, n)$ is clearly located somewhere between NEXPTIME and EXPSPACE and allow us to provide more refined upper and lower bounds for the logic.

3 Basic facts and expressivity examples

This section will be dedicated to proving useful facts about QCTL_{EX} as well as showing what kind of properties are expressible in the logic.

3.1 Shallow tree models

As in the case of classical modal logic we establish *shallow model property* for QCTL_{EX} . The construction is standard and goes in exactly the same way as in the literature (see e.g. Theorem 2.3 from [4]). Thus the following lemma holds:

Lemma 3.1. *Every satisfiable QCTL_{EX} formula φ has a tree model of a depth bounded by a modal depth $\text{md}(\varphi)$.*

Due to the above lemma it is enough to consider models to be finite trees rather than possibly infinite unfoldings of arbitrary Kripke structures. We will make this assumption and from now models will be always finite trees.

3.2 Prenex Normal Form

It is known (e.g. Proposition 3.1 in [16]) that any QCTL formula can be transformed into equisatisfiable formula in Prenex Normal Form. We revisit that result and reprove it for QCTL_{EX} . Since our logic does not contain Until operator and models are finite trees, we can employ a technically simpler transformation than proposed in [16]. An another advantage of our translation process is that it could not increase the number of alternations of quantifiers in an input formula (which is not the case in [16]).

Our rewriting system consists of three following rules:

- $Qx.\varphi \equiv \overline{Q}x.\neg\varphi$,
- $Q_1x_1.\varphi_1 \vee Q_2x_2.\varphi_2 \equiv Q_1x_1.Q_2x_2.\varphi_1 \vee \varphi_2$, and
- $\text{EX}(Qx.\varphi) \equiv Qx.\text{EX}(\varphi)$,

where Q is a propositional quantifier, $\overline{Q}$ its dual to Q and φ is quantifier free.

The following lemma is routine:

Lemma 3.2. Any QCTL_{EX} formula φ can be transformed into equisatisfiable formula in Prenex Normal Form, which length is bounded linearly in the size of φ .

From now we will assume that an input formula is always given in Prenex Normal Form.

3.3 Uniqueness and Counting Ability

Using simple bisimulation arguments one can prove that there is no CTL formulae distinguishing a tree model $\mathfrak{T}$ and a tree $\mathfrak{T}'$ where every node from $\mathfrak{T}$ is copied twice. Therein, we will show that it is no longer true in QCTL_{EX} i.e. we provide a formula enforcing that a given property φ holds at exactly one successor.

It is easy to see that such formula can be written in the following way:

$$\mathbf{EX}_{=1}(\varphi) \stackrel{\text{def}}{=} \forall p \mathbf{EX}(\varphi) \wedge \neg(\mathbf{EX}(p \wedge \varphi) \wedge \mathbf{EX}(\neg p \wedge \varphi))$$

macro:
 $\mathbf{EX}_{=1}(\varphi)$

Another simple property, not expressible in QCTL, is counting. To write down formula, specifying that at least n successors satisfy some property φ , one can simply enumerate them using unary predicates $p_0, \dots, p_{n-1}$ as in the following example:

$$\mathbf{EX}_{\geq n}(\varphi) \stackrel{\text{def}}{=} \exists p_0 \exists p_1 \dots \exists p_{n-1} \bigwedge_{i=0}^{n-1} \mathbf{EX}(p_i \wedge \varphi) \wedge \bigwedge_{0 \leq i < j \leq n-1} \neg \mathbf{EX}(p_i \wedge p_j \wedge \varphi)$$

macro:
 $\mathbf{EX}_{\geq n}(\varphi)$

The previous formulae can be used to define so-called *graded modalities* $\mathbf{EX}_{\geq K}$, $\mathbf{EX}_{\leq K}$ and $\mathbf{EX}_{=K}$, which their natural interpretation. An ability to define grading in QCTL_{EX} link our logic with well-known graded modal logic [11] and provide us a neat expressivity result.

3.4 Local nominals

Hybrid logics is another expressive extension of ordinary modal logic, which extends the logic with devices for naming worlds and for accessing them by their names. We will focus on three of them: *nominals* $x, y, z, \dots$ (aka constants), a *binder* $\downarrow$ and a *satisfaction operator* $@$. Nominals can be seen as a names - they are true at exactly one world of a model and nowhere else. Then, during evaluation of a formula, a binder $\downarrow_x$ can be used to give a name to the current world, while $@_x \varphi$ says that in the world labeled with name x some property φ holds (without jumping directly to it).

We stress here that these three constructs are powerful enough to make hybrid logic undecidable. In contrast, when we allow only nominals and the satisfaction operator, obtained logic is PSPACE-complete. For a survey on the complexity issues with hybrid logics see [1].

3.4.1 Informal introduction

Due to simple reasons e.g. that depth of our models can be bounded or that we cannot jump into an arbitrary node in a tree, our logic QCTL cannot capture hybrid logics. Nevertheless, therein we will introduce a novel toolkit which will mimic constructs from the latter logic. We will proudly refer to this technique as *local nominals*. The word local in the name of the technique means that we are able to quantify over nodes only in the limited scope i.e. fixed number of steps.

3.4.2 Defining "at" operator and binders

We will use binders $\downarrow_x^k$ to set up a uniquely identified node labeled with x exactly k steps below. To access such a node we will use a macro $@_x^k$, which will make formulae shorter and more readable. Note that the definitions below make sense only if binders and "at" operators are in the scope of propositional quantifiers.

We start with defining $@_x^1$ and $\downarrow_x^1$ formulae:

macros: $@_x^1, \downarrow_x^1$

$$\downarrow_x^1 \stackrel{\text{def}}{=} \mathbf{EX}_{=1}(x), \quad @_x^1(\varphi) \stackrel{\text{def}}{=} \mathbf{EX}(x \wedge \varphi)$$

To define local nominals for a higher number of steps, we need to provide a unique path from a current node to the nominal. It can be done by introducing fresh local nominals, called e.g. $path_{to_x}$ leading to a proper target.

macros: $@_x^k, \downarrow_x^k$

$$\downarrow_x^{k+1} \stackrel{\text{def}}{=} \exists path_{to_x} \downarrow_{path_{to_x}}^1 \wedge @_x^1(path_{to_x} \downarrow_x^k),$$

$$@_x^{k+1}(\varphi) \stackrel{\text{def}}{=} @_x^1(path_{to_x} (@_x^k(\varphi)))$$

We stress here that we do not really require that a unique node is labeled with a name x . For our purposes it will be enough if such a node is uniquely identified. Additionally we may want to quantify over pair of nodes, so $\downarrow_{x,y}^k$ will be the same as

$$\downarrow_{x,y}^k \stackrel{\text{def}}{=} \downarrow_x^k \wedge \downarrow_y^k \wedge @_x^k(\neg y) \wedge @_y^k(\neg x)$$

3.4.3 A toy example

We would like to illustrate the technique on the following example. Imagine that we want to write a formula which is satisfied at a given node if and only if there is no pair of its successors such they agree on every atomic proposition from a set AP. Using $\downarrow$ and $@$ notation, formula can be written in the following way:

$$\varphi_{uniqueAP} \stackrel{\text{def}}{=} \underbrace{\forall x \forall y \downarrow_{x,y}^1 \rightarrow}_{\text{name two different successors}} \neg \left(\underbrace{\bigwedge_{a \in AP} @_x^1(a) \leftrightarrow @_y^1(a)}_{\text{agree on AP}} \right)$$

Note that $\varphi_{uniqueAP}$ can be written in graded modal logic easily, but likely requires exponential size (iterate over all possible combinations of letters from AP).

4 Bounded satisfiability

In this section we will study the satisfiability problem for QCTL_{EX} over trees with bounded degree. We will show its membership to $\text{STA}(*, 2^{\mathcal{O}(n)}, n)$, a less-known complexity class which lies between NEXPTIME and EXSPACE . It contrasts with TOWER -hardness of the full logic over binary trees from [16]. We will also provide NEXPTIME lower bound for the case of binary trees. We believe that satisfiability problem is complete for $\text{STA}(*, 2^{\mathcal{O}(n)}, n)$ in the bounded case, although we will not present the proof here.

4.1 Proving a lower bound

We start with proving a NEXPTIME lower bound. To do that, we will first introduce a convenient tiling problem and then encode it inside QCTL_{EX}.

4.1.1 Tiling problem

A similar formulation of the problem can be found in [18].

A *tiling system* is a tuple $S = (T, H, V)$, where T is a finite set of *tiles*, $H \subseteq T \times T$ and $V \subseteq T \times T$ are, respectively, the horizontal and vertical adjacency relations. A *tiling problem* instance is a tuple $(S, n, top_0, top_1, \dots, top_{n-1}, bot)$, where $n \in \mathbb{N}$ is a natural number, $top_i \in T$ are initial tiles and $bot \in T$ is a *final* tile. A *square tiling* of a grid $G = 2^n \times 2^n$ is a mapping $\tau : G \rightarrow T$. A *proper square tiling* is a tiling, where mapping τ respect the horizontal and vertical rules. In *square tiling problem* we ask whether there exists a proper square tiling of a grid $2^n \times 2^n$, such that first n tiles in the top row are $top_0, top_1, \dots, top_{n-1}$ (in the same order) and the tile bot appear somewhere at the bottom row.

The following theorem is classic and can be found in multiple papers.

Theorem 4.1 ([22]). *Square tiling problem is NEXPTIME-complete.*

4.1.2 Working with number encodings

For a given capital letter A , by $\exists A_n$ we will denote the block of n existential quantifiers with predicates a_i , namely $\exists_{a_{n-1}} \exists_{a_{n-2}} \dots \exists_{a_0}$. We will use such predicates, to encode natural numbers in the following way - the predicate a_i will be responsible of the i -th bit of the corresponding number. The least significant bit will be denoted with a_0 . We define $\forall A_n$ analogously. When we consider a tree model $\mathfrak{T}$, its node v and set of n predicates A_n , we will say that world w satisfies $A_n=k$, if and only if, the number represented by $a_{n-1}, a_{n-2}, \dots, a_0$ is equal to k (and analogously for an inequality symbol). For example a macro $A_4=5$ will be expanded as $\neg a_3 \wedge a_2 \wedge \neg a_1 \wedge a_0$. The last macro we will introduce here will say that the number encoded with predicates A_n is equal to number encoded with predicates B_n . We will denote it as $A_n=B_n$, with an obvious unfolding $\bigwedge_{i=0}^{n-1} a_i \leftrightarrow b_i$.

macro: $\exists A_n$
macro: $\forall A_n$
macro: $A_n=k$
macro:
 $A_n=B_n$

4.2 Arithmetical relations between two number encodings

Given two local nominals called x and y located on level lvl , labeled with predicates A_n , we would like to express some arithmetical relations between the numbers they encode. We start with a simply formula stating that two nominals carries the same number written on A_n predicates.

macro:
 $\varphi_=(x, y, lvl, A_n)$

$$\varphi_=(x, y, lvl, A_n) \stackrel{\text{def}}{=} \bigwedge_{i=0}^{n-1} @_x^{lvl}(a_i) \leftrightarrow @_y^{lvl}(a_i).$$

Then, we turn our attention to more advanced constraints on the number representations. We first describe how to define $\varphi_>(x, y, lvl, A_n)$ formula, specifying that the value represented by a local nominal x is greater than the value of local nominal y . The relation " $>$ " holds between x and y , if there is $0 \leq i < n$ such these two numbers differs on i th bit (i th bit in x is turned on and in y turned off) and they agree on all later bits. It can be written in the following way:

macro:
 $\varphi_>(x, y, lvl, A_n)$

$$\varphi_{>}(x, y, lvl, A_n) \stackrel{\text{def}}{=} \bigvee_{i=0}^{n-1} \left(@_x^{lvl}(a_i) \wedge @_y^{lvl}(\neg a_i) \wedge \underbrace{\left(\bigwedge_{j=i+1}^{n-1} @_x^{lvl}(a_j) \leftrightarrow @_y^{lvl}(a_j) \right)}_{\text{rewrite other bits}} \right)$$

The second formula $\varphi_{+1}(x, y, lvl, A_n)$, specifying that y is an immediate successor of x is a little harder to write. We will find the first zero bit i in x . Since it's the first zero bit, thus every bit on the right of i should be turned on. In y the corresponding bits should be reseted and i th bit should be turned on. The rest of bits should coincide on x and y .

macro:
 $\varphi_{+1}(x, y, lvl, A_n)$

$$\bigwedge_{i=0}^{n-1} \left(\underbrace{@_x^{lvl} \left(\neg a_i \wedge \bigwedge_{j=0}^{i-1} a_j \right)}_{\text{look for the first zero bit}} \rightarrow \left(\underbrace{@_y^{lvl} \left(\bigwedge_{j=0}^{i-1} \neg a_j \wedge a_i \right)}_{\text{reset previous bits, set } a_i} \wedge \underbrace{\bigwedge_{j=i+1}^{n-1} @_x^{lvl}(a_j) \leftrightarrow @_y^{lvl}(a_j)}_{\text{rewrite other bits}} \right) \right)$$

The presented formulae are straightforward and since they belong to mathematic folklore we will skip their proofs (see e.g. [14]). The only remark is that by using their boolean combinations one can easily define other thresholds like $<$, $\leq$ or $>$. Similarly, by iterating the *next* relation multiple times, it is possible to define $+k$ relation for any natural number k .

4.3 Encoding an exponential grid

Let us fix a natural number $n \in \mathbb{N}$. To reduce a tiling problem to our logic, we would like to find a way to encode an exponential grid of size $2^n \times 2^n$. To do that, each node on a certain level of a tree will carry a distinct pair of H_n and V_n predicates, which will uniquely identify a corresponding cell in a grid with its horizontal and vertical position.

The formula φ_{grid} will consist of three parts. First of all, we will enforce that there is a node labeled with $H_n=0$ and $V_n=0$ at the level $2n$ (since such level will guarantee that over binary tree there is enough space for all cells). Then, we will use $\varphi_{+1}(\cdot, \cdot, \cdot, \cdot)$ formula, to say that every non-final cell has a vertical and a horizontal successor. Finally, it will be enough to specify that there is no two different nodes at level $2n$ carrying exactly the same H_n and V_n position.

$$\varphi_{grid} \stackrel{\text{def}}{=} \exists V_n \exists H_n \varphi_{firstCell} \wedge \varphi_{vSuccessors} \wedge \varphi_{hSuccessors} \wedge \varphi_{uniqueness}$$

To define the first cell we simply put a nominal in the distance $2n$ with value 0 on both coordinates.

$$\varphi_{firstCell} \stackrel{\text{def}}{=} \exists x_{fst} \downarrow_{x_{fst}}^{2n} \wedge @_{x_{fst}}^{2n} (H_n=0 \wedge V_n=0)$$

Now we turn our attention to define a formulae that every non-final cell in a grid has a proper H and V successors. To do that, we first specify two relations which holds between two local nominals x and y on level $2n$ if and only if, y is respectively H or V successor of a cell represented by x .

macro:
 $\varphi_{succH}(x, y)$

$$\varphi_{succH}(x, y) \stackrel{\text{def}}{=} \varphi_{+1}(x, y, 2n, H_n) \wedge \varphi_{=}(x, y, 2n, V_n)$$

macro:
 $\varphi_{succV}(x, y)$

$$\varphi_{succV}(x, y) \stackrel{\text{def}}{=} \varphi_{+1}(x, y, 2n, V_n) \wedge \varphi_{=}(x, y, 2n, H_n)$$

Using above formulae it is easy to define that every non-final cell has a proper successor. It can be done in the following way:

$$\varphi_{h\text{Successors}} \stackrel{\text{def}}{:=} \forall x \downarrow_x^{2n} \wedge @_x^{2n}(H_n \neq 2^n - 1) \rightarrow \exists y_h \downarrow_{y_h}^{2n} \wedge \varphi_{\text{succ}H}(x, y_h)$$

$$\varphi_{v\text{Successors}} \stackrel{\text{def}}{:=} \forall x \downarrow_x^{2n} \wedge @_x^{2n}(V_n \neq 2^n - 1) \rightarrow \exists y_v \downarrow_{y_v}^{2n} \wedge \varphi_{\text{succ}V}(x, y_v)$$

The last formula says that there is no two cells at the same position.

$$\varphi_{\text{uniqueness}} \stackrel{\text{def}}{:=} \forall x \forall y \downarrow_{x,y}^{2n} \rightarrow \neg (\varphi_{=}(x, y, 2n, H_n) \wedge \varphi_{=}(x, y, 2n, V_n))$$

4.4 Encoding tiles

Using the formula φ_{grid} from the previous section we were able to encode an exponential square grid inside our logic. Now, we would like to take an arbitrary instance of tiling problem and produce a formula from QCTL_{EX} equivalent to it. Since this task is rather routine after introducing all techniques in the previous section, we will present the further steps in the Appendix [A](#).

Thus we get:

Theorem 4.2. *A satisfiability problem for QCTL_{EX} is NEXPTIME-hard over binary trees.*

4.5 Proving an upper bound

Now we are going to focus on the upper part of the proof, which seems to be much easier to deal with than the lower bound.

Let us stress here that the size of expected model can be bounded. Let φ be a formula from QCTL_{EX} written in Prenex Normal Form. First of all, since we use only **EX** modality, the depth of a tree model can be bounded with $|\varphi|$. Additionally, we are interested in the case when every node in a tree has a bounded degree with some fixed number d , thus we obtain a bound $1 + d + d^2 + \dots + d^{|\varphi|} \leq d^{|\varphi|+1}$ on a tree model. Note that this number is exponential in $|\varphi|$.

Corollary 4.2.1. *Let φ be QCTL_{EX} formula φ and let $d \in \mathbb{N}$. Then if a formula φ is satisfiable over trees where every node has its degree bounded with d , then φ has also a model of size smaller than $d^{|\varphi|+1}$.*

Note that this corollary immediately gives us an algorithm to test satisfiability. We can first guess the tree structure $\mathfrak{T}$ and then check all possible valuations given with propositional quantifiers and check whether $\mathfrak{T}$ is indeed a model. All this process can be done in EXPSpace, since we keep only an exponential tree in the memory.

A presented algorithm can be slightly improved to obtain a tighter complexity class $\text{STA}(*, 2^{\mathcal{O}(n)}, n)$. We first transform the formula into Prenex Normal Form. Then, we guess the model. For every quantifier in the formula, we guess the labeling of the whole structure at once. Since we can test in PTIME whether a given tree is a model of a CTL_{EX} formula, the whole algorithm works in EXPTIME with only linear number of alternations. Thus we get $\text{STA}(*, 2^{\mathcal{O}(n)}, n)$ upper bound and the following theorem.

Theorem 4.3. *For every QCTL_{EX} formula φ and natural number $d \in \mathbb{N}$ it is decidable in $\text{STA}(*, 2^{\mathcal{O}(n)}, n)$ whether a formula is satisfiable over d -bounded trees or not.*

4.6 A remark on the simplest possible case

All proofs from the previous section have a big advantage because they work for every possible bound $d \geq 2$ on degree of nodes in tree models. However, we omitted a simple case where $d = 1$ and the models are not really trees but words.

In case when $d = 1$, one can use Shallow Model Property and since our structure is not branching, the total size is polynomial in the size of an input formula. Thus, we can guess the expected model in polynomial space, rewrite formula into Prenex Normal Form and enumerate all possible valuations of propositional quantifiers to we check whether a guessed structure is indeed a model. That gives us a PSPACE upper bound on the problem. The lower bound comes trivially from QBF, which is a standard PSPACE-complete problem.

Theorem 4.4. *The satisfiability problem for QCTL_{EX} over words is PSPACE-complete.*

5 General satisfiability

In this section we will focus on the main result of the internship i.e. that satisfiability problem for QCTL_{EX} is TOWER-hard. Instead of giving a detailed construction, we will rather focus on giving proof ideas and general overview of the construction. The full proofs will be available in the published version of the paper.

5.1 Overview of the method

We already know that the depth of a tree model can be bounded linearly in the size of an input formulae. It means that to obtain some hardness result better than EXPSpace from the previous section, we need to use the fact that degree of a single node can be arbitrary big. In the next few lines we will present the general idea behind enforcing a huge degree of a single node and how it can be achieved with local nominals technique.

We first define a tower function $t(n, k)$. In the basic case $t(n, 1) = 2^n$. Then $t(n, k + 1) = 2^{t(n, k)}$. For the rest of the report assume than n is fixed.

We will divide our tree into *layers*. Leaves of the tree will be at layer 0, their parents at layer 1, their parents at layer 2 and so on. Each node at layer k will be labeled with a predicate val_k . We define the *number* of a node i.e. number represented by binary encoding of val_{k-1} predicates on its children. Then, we will say that at each layer there is a node representing number 0 and for each node with value x , we have also a node with value $x + 1$. By employing $+1$ relation we obtain every number between 0 and $t(n, k) - 1$. We will use the previous techniques on arithmetical encoding from section about bounded satisfiability.

The problem with that kind of construction occurs even at the second level i.e. when we are trying to enforce doubly-exponential degree. Using standard methods, to encode doubly-exponential number, we will need exponentially many bit predicates, which is not the case here, since we require formulae to have polynomial size. We overcome that difficulty with proposing a different method of adding one to the number based on *lsr*-partitioning and arithmetical relations on lower layers.

5.2 Sketch of a general approach

Below we present the complete list of formula which will be used in the proof later.

Formula	Requirement	Intuitive meaning
$\varphi_{type}^{(k)}$	-	node has exactly $t(n, k)$ successors
$\varphi_{first}^{(k)}$	interpreted at a node at level k	node has number 0
$\varphi_{last}^{(k)}$	interpreted at a node at level k	node has number $t(n, k) - 1$
$\varphi_{uniqueness}^{(k)}$	interpreted at a node at level k	there are no two children with the same number
$\varphi_{populate}^{(k)}$	interpreted at a node at level k	each children has its +1-successor
$\varphi_{=}^{(k)}(x, y)$	interpreted at a node at level $k + 1$	two local nominals carries the same number
$\varphi_{+1}^{(k)}(x, y)$	interpreted at a node at level $k + 1$	two local nominals enjoy +1 relation of their numbers
$\varphi_{>}^{(k)}(x, y)$	interpreted at a node at level $k + 1$	two local nominals enjoy > relation of their numbers

The general schema for writing down $\varphi_{type}^{(k)}$ formula i.e. the formula enforcing $t(n, k)$ degree will be the following:

$$\varphi_{type}^{(k)} \stackrel{\text{def}}{=} \exists val_{k-2} \mathbf{AX}(\varphi_{type}^{(k-1)}) \wedge \mathbf{EX}(\varphi_{first}^{(k-1)}) \wedge \varphi_{uniqueness}^{(k)} \wedge \varphi_{populate}^{(k)}$$

We first label the whole model with val_{k-2} predicates. They will be used to describe the number represented with a given node. Then we specify that all the successors lies on the lower level i.e. all satisfy $\varphi_{type}^{(k-1)}$ formula. Then, we enforce that there is a node representing number 0 (i.e. $\varphi_{first}^{(k-1)}$ formula), all numbers are unique (there are no two nodes carrying the same number) – $\varphi_{uniqueness}^{(k)}$ formula and finally that all successors fill the gaps between number 0 and $t(n, k) - 1$ – the formula $\varphi_{populate}^{(k)}$.

5.3 The first level - a detailed construction

In this section we will focus on the very first level of the construction.

Let us explain first how we will define $\varphi_{type}^{(1)}$ formula. Each children of a node satisfying $\varphi_{type}^{(1)}$ will be a leaf labeled with some combination of P_n predicates (to read its number) as well as predicate val_0 (which will be used at next levels of the construction). We will specify that there is a leaf with number 0 written as binary number represented at P_n predicates and each node has its successor with +1 value. The whole resembles hardness proofs from the previous section, so we will omit the proofs.

$$\varphi_{type}^{(1)} \stackrel{\text{def}}{=} \exists P_n \mathbf{AX}(\varphi_{type}^{(0)}) \wedge \mathbf{EX}(\varphi_{first}^{(0)}) \wedge \varphi_{uniqueness}^{(1)} \wedge \varphi_{populate}^{(1)}$$

The above formula is very easy to understand. To specify that each successor is at layer 0 we simply define that a node is a leaf: $\varphi_{type}^{(0)} \stackrel{\text{def}}{=} \neg \mathbf{EX}(\top)$. To say that a node at layer 0 carries value 0 we simply say: $\varphi_{type}^{(0)} \stackrel{\text{def}}{=} \bigwedge_{i=0}^{n-1} \neg p_i$. Uniqueness is a little harder to define. We simply want to say that there is no children with the same number. It can be done it this way: $\varphi_{uniqueness}^{(1)} \stackrel{\text{def}}{=} \forall x, y \downarrow_{x,y}^1 \rightarrow \neg \varphi_{=}^{(0)}(x, y)$, where $\varphi_{=}^{(0)}(x, y)$ is simply equal to $\varphi_{=}^{(0)}(x, y) \stackrel{\text{def}}{=} \bigwedge_{i=0}^{n-1} @_x^1(p_i) \leftrightarrow @_y^1(p_i)$. The last formula $\varphi_{populate}^{(1)}$ says that each node has its proper successor $\forall x \downarrow_x^1 \wedge @_x^1(\neg \varphi_{last}^{(0)}) \rightarrow \exists y \downarrow_y^1 \wedge \varphi_{+1}^{(0)}(x, y)$, where $\varphi_{+1}^{(0)}(x, y)$ is defined in exactly the same way as in the previous section i.e. $\varphi_{+1}(x, y, 1, P_n)$. We also define $\varphi_{>}^{(1)}(x, y)$ as $\varphi_{>}(x, y, 1, P_n)$.

5.4 Going higher - the second level

The general scheme for defining the second level will be the same as previously, that is:

$$\varphi_{type}^{(2)} \stackrel{\text{def}}{=} \exists val_0 \mathbf{AX}(\varphi_{type}^{(1)}) \wedge \mathbf{EX}(\varphi_{first}^{(1)}) \wedge \varphi_{uniqueness}^{(2)} \wedge \varphi_{populate}^{(2)}$$

We have already defined $\varphi_{type}^{(1)}$ formulae, so we know that all the successors of the target node will have exactly $t(n, 1) = 2^n$ successors. Each of these successors will be labeled with val_0 predicate, so each they will represent a big number written on 2^n bits. We would like to use that fact and perform arithmetical operation on such encodings.

We start with defining $\varphi_{first}^{(1)}$ formula. It is enough to say that all val_1 predicates are switched off. It can be done with a formula:

$$\varphi_{first}^{(1)} \stackrel{\text{def}}{=} \mathbf{AX}(\neg val_0)$$

Then, to define $\varphi_{uniqueness}^{(2)}$ we simply say that there are no two nodes with the same number i.e.

$$\varphi_{uniqueness}^{(1)} \stackrel{\text{def}}{=} \forall x, y \downarrow_{x,y}^1 \rightarrow \neg \varphi_{=}^{(1)}(x, y),$$

where $\varphi_{=}^{(1)}(x, y)$ will be defined later.

Last, but not least, we define $\varphi_{populate}^{(2)}$ formula. It is enough to say that for each node there is a proper successor with +1 value. It can be done in the following way:

$$\forall x \downarrow_x^1 \wedge @_x^1(\neg \varphi_{last}^{(1)}) \rightarrow \exists y \downarrow_y^1 \wedge \varphi_{+1}^{(1)}(x, y),$$

where $\varphi_{last}^{(1)}$ is simply

$$\varphi_{last}^{(1)} \stackrel{\text{def}}{=} \mathbf{AX}(val_0).$$

The only problematic part is to define proper arithmetical operations, which will be the main goal of the next section.

5.5 Arithmetical encoding at the second level

Let us remind how we employed +1 relation at the first level. First of all we were looking for the first zero bit in x . We will refer to it as *selector*. Then we know that selector bit in x must be turned off, while all the bits *on the right of selector* should be turned on. Meanwhile the selector in y should be turned on and all the bits on the right of selector should be zeroes. The only missing part is to ensure that all the bits on the left of selector bits coincide in both x and y .

Such description give us a high-level algorithm for adding one to a given number. Since we cannot select a bit by enumerating predicates (since there exponentially many successors at the first level), we will use a predicate s (which stands for *selector*) to simulate that process. Then we will use l and r predicates to specify what is on the right and on the left of s . Finally, we will implement reseting bits, rewriting bits and setting all bits true. We will obtain desired formula with conjunction of the following properties:

- A There is exactly one selector
- B Every node at level 0 is labeled either by s , l or r and exactly one of these predicates holds
- C Predicate val_0 in selector is turned on for y and turned off for x
- D Predicates val_0 for right nodes for x (resp. y) is always 1 (resp. 0)
- E Nodes labeled with r have smaller number than selector and nodes labeled with l have greater number than selector

F All values of val_0 for left nodes coincide for x and y .

The first property is very easy to write down. We use $\mathbf{EX}$ to specify that such a node exist and using local nominals we enforce that any two nodes w and w' carrying predicate s must have the same number.

$$\varphi_A \stackrel{\text{def}}{=} @_x^1(\mathbf{EX}_{=1}(s)) \wedge @_y^1(\mathbf{EX}_{=1}(s))$$

The second property defines a partition of a universe. Writing down the formula is rather straightforward. We will give one only for x , since for y it will be analogous.

$$\varphi_B = @_1^x(\mathbf{AX}(s \vee l \vee r) \wedge \neg \mathbf{EX}(s \wedge l) \wedge \neg \mathbf{EX}(s \wedge r) \wedge \neg \mathbf{EX}(l \wedge r))$$

The third and fourth formulae can be written in the following way:

$$\varphi_C = @_1^x(\mathbf{AX}(s \rightarrow \neg val_0)) \wedge @_1^y(\mathbf{AX}(s \rightarrow \neg val_0))$$

$$\varphi_D = @_1^x(\mathbf{AX}(r \rightarrow val_0)) \wedge @_1^y(\mathbf{AX}(r \rightarrow \neg val_0))$$

The fifth formula is the first challenging one. We will choose the selector w and one of left labeled elements w' and then use the $w > w'$ property on the lower level defined in the previous sections. Again, we will write the formula only for x and elements labeled with r , since the proper formula for y and left nodes is analogous.

$$\varphi_E = @_x^1(\forall w \forall w' \downarrow_{w,w'}^1 \wedge @_w^1(s) \wedge @_{w'}^1(r) \rightarrow \varphi_{>}^{(1)}(w, w'))$$

For the last formula for each left-labeled node from x we will chose a proper witness in y and ensure that such a witness is proper.

$$\varphi_F = \forall w @_x^1(\downarrow_w^1) \rightarrow \exists w' @_y^1(\downarrow_{w'}^1) \wedge \bigwedge_{p \in P_n \cup \{val_0\}} @_x^1(@_w^1(p)) \leftrightarrow @_y^1(@_{w'}^1(p))$$

We define $\varphi_{+1}^{(1)}(x, y)$ as a conjunction of all previous formulae. A formula $\varphi_{\leq}^{(1)}(x, y)$ can be defined in the similar way. Due to space limits we omit the proof that the presented formulae really works.

Our construction can be generalized to handle arithmetical relation on higher levels. To employ $+1$ relation we simply use partitioning multiple times and employ $<$ and $+1$ on the lower levels using presented recipe. Because the construction is very technical we will not present it here.

5.6 From huge degree to hardness

Once we enforce trees to have a “large” i.e. k -exponential degree, we can employ the reduction from $\mathbf{t}(n, k) \times \mathbf{t}(n, k)$ tiling systems from [22], which are known to be k -EXPTIME complete. The proof ideas are simiar to the case of bounded degree, when we used exponential tillings.

6 Expressive power of QCTL_{EX}

We have already proved in Section 3.2 that any QCTL_{EX} can be turned into Prenex Normal Form. Here we will consider the expressive power of QCTL_{EX} with bounded number of alternations of propositional quantifiers.

Definition 6.1. For a given $n \geq 1$, a QCTL_{EX} sentence is said to be Σ_i (resp. Π_i) if its prenex normal form has n blocks of nested quantifiers (or, equivalently, $(n-1)$ alternations) and starts with an $\exists$ (resp. $\forall$) quantifier. Since these classes are not closed under complement, we will consider also $\mathcal{BS}\Sigma_i$ sentences i.e. boolean combinations of Π_i and Σ_i levels.

It is easy to see that Σ_1 and Π_1 are more expressive than standard CTL_{EX} . To do that, one can consider the formula $\mathbf{EX}_{=1}a$ (see: Section 3.3) together with its negation and employ standard bisimulation arguments. To show that $\mathcal{BS}\Sigma_1$ is more expressive, we can employ the formula $\mathbf{EX}_{=1}(\mathbf{EX}_{=1}(a))$. The non-definability proof of such a formula can be found in [8].

The separation of higher levels of alternation hierarchy seems to be a very natural yet challenging task. Especially it seems that higher levels of alternation hierarchy gives us more power but it is not the case in the full logic [8].

We tried to separate Σ_i and Π_i classes with formulae $\mathbf{EX}_{=1}(\mathbf{EX}_{=1}(\mathbf{EX}_{=1}(\dots(\mathbf{EX}_{=1}(a))))$ or a similar one. All of them lies inside so-called *graded* CTL_{EX} logic (again, see: Section 3.3) i.e. an extension of QCTL_{EX} with $\mathbf{EX}_{=K}$, $\mathbf{EX}_{\leq K}$ and $\mathbf{EX}_{\geq K}$ for any natural number K . We will show that separation with these kind of formulae is not possible due to fact that every graded CTL_{EX} formula is expressible at Σ_2 level.

Lemma 6.1. *Any graded CTL_{EX} property can be expressed in Σ_2 .*

(idea). Consider a graded CTL_{EX} formula φ . Without loss of generality, we can assume that only modalities used in φ are of the form $\mathbf{EX}_{\geq k}$ for some natural number k . Let h_φ be a maximal number of nested $\mathbf{EX}$ in the formula φ . We will call such a number by the *height* of φ .

Let ψ be innermost subformula of φ containing a modal operator. Thus ψ is of the form $\mathbf{EX}_{\geq k}\psi'$ for some modality-free formula ψ' . We will introduce a new atomic proposition p and write a proper formula, such that p holds on a given node if and only if ψ holds. Of course we cannot do it for the whole tree model, but it is enough to ensure that such condition is satisfied up to depth h_φ . Let φ' be $\exists p \varphi[\psi/p] \wedge \bigwedge_{i=0}^{h_\varphi} \mathbf{AX}^i(p \leftrightarrow \mathbf{EX}_{\geq k}\psi')$. We repeat the process, until the original formula φ becomes modality-free. Then, we unfold the definition of $\mathbf{EX}_{\geq k}$ using the one presented in Section 3.3. We denote obtained formula by φ_{fin} . Easy manipulation of quantifiers shows that the obtained formula is in Σ_2 .

It is rather easy to see that φ and φ_{fin} are equisatisfiable in the following sense: for every tree $\mathfrak{T}$ and its root v we have that $\mathfrak{T}, v \models \varphi$ if and only if $\mathfrak{T}, v \models \varphi_{\text{fin}}$. □

7 Conclusions

During the internship we focus on providing the complexity bounds for QCTL_{EX} . We proved that the bounded satisfiability problem is in $\text{STA}(*, 2^{\mathcal{O}(n)}, n)$ and NEXPTIME-hard by using the technique of local nominals and by encoding the exponential tiling problems inside the logic. We also proved that the satisfiability problem for QCTL_{EX} is TOWER-hard (i.e. QCTL retains its lower bound even if we drop until operator). In this report we presented a draft of the proof. Additionally, we provided a useful fact that graded QCTL_{EX} lies inside Σ_2 level, so there is no hope for separating expressive power of Σ_i and Π_i levels by using only graded modalities.

For the future work we will focus on the improvements on the presentation side of hardness result for general satisfiability problem. The whole construction is very technical and difficult to read and understand. We hope that it can be improved in the nearest future. We plan to submit the full version of the report to LICS 2019 conference.

References

- [1] Carlos Areces, Patrick Blackburn, and Maarten Marx. A road-map on complexity for hybrid logics. In Jörg Flum and Mario Rodríguez-Artalejo, editors, *Computer Science Logic, 13th International Workshop, CSL '99, 8th Annual Conference of the EACSL, Madrid, Spain, September 20-25, 1999, Proceedings*, volume 1683 of *Lecture Notes in Computer Science*, pages 307–321. Springer, 1999. [doi:10.1007/3-540-48168-0_22](https://doi.org/10.1007/3-540-48168-0_22).
- [2] Sanjeev Arora and Boaz Barak. Computational complexity - a modern approach, 2009.
- [3] Leonard Berman. The complexity of logical theories. *Theor. Comput. Sci.*, 11:71–77, 1980. [doi:10.1016/0304-3975\(80\)90037-7](https://doi.org/10.1016/0304-3975(80)90037-7).
- [4] Patrick Blackburn, Maarten de Rijke, and Yde Venema. *Modal Logic*. Cambridge Tracts in Theoretical Computer Science. Cambridge University Press, 2001. [doi:10.1017/CB09781107050884](https://doi.org/10.1017/CB09781107050884).
- [5] Ashok K. Chandra, Dexter Kozen, and Larry J. Stockmeyer. Alternation. *J. ACM*, 28(1):114–133, 1981. [doi:10.1145/322234.322243](https://doi.org/10.1145/322234.322243).
- [6] Edmund M. Clarke and E. Allen Emerson. Design and synthesis of synchronization skeletons using branching-time temporal logic. In Dexter Kozen, editor, *Logics of Programs, Workshop, Yorktown Heights, New York, USA, May 1981*, volume 131 of *Lecture Notes in Computer Science*, pages 52–71. Springer, 1981. [doi:10.1007/BFb0025774](https://doi.org/10.1007/BFb0025774).
- [7] Dennis Dams, Rob Gerth, and Orna Grumberg. Abstract interpretation of reactive systems. *ACM Trans. Program. Lang. Syst.*, 19(2):253–291, 1997. [doi:10.1145/244795.244800](https://doi.org/10.1145/244795.244800).
- [8] Amélie David, François Laroussinie, and Nicolas Markey. On the expressiveness of QCTL. In *CONCUR*, volume 59 of *LIPIcs*, pages 28:1–28:15. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, 2016.
- [9] Stéphane Demri and Raul Fervari. On the complexity of modal separation logics. In Guram Bezhanishvili, Giovanna D’Agostino, George Metcalfe, and Thomas Studer, editors, *Proceedings of the 10th Conference on Advances in Modal Logics (AiML’18)*, Bern, Switzerland, August 2018. College Publications. To appear.
- [10] E. Allen Emerson and A. Prasad Sistla. Deciding full branching time logic. *Information and Control*, 61(3):175 – 201, 1984. [doi:https://doi.org/10.1016/S0019-9958\(84\)80047-9](https://doi.org/10.1016/S0019-9958(84)80047-9).
- [11] Kit Fine. In so many possible worlds. *Notre Dame J. Formal Logic*, 13(4):516–520, 10 1972. [doi:10.1305/ndjfl/1093890715](https://doi.org/10.1305/ndjfl/1093890715).
- [12] Tim French. Decidability of quantified propositional branching time logics. In Markus Stumptner, Dan Corbett, and Michael J. Brooks, editors, *AI 2001: Advances in Artificial Intelligence, 14th Australian Joint Conference on Artificial Intelligence, Adelaide, Australia, December 10-14, 2001, Proceedings*, volume 2256 of *Lecture Notes in Computer Science*, pages 165–176. Springer, 2001. [doi:10.1007/3-540-45656-2_15](https://doi.org/10.1007/3-540-45656-2_15).
- [13] Tim French. Quantified propositional temporal logic with repeating states. In *10th International Symposium on Temporal Representation and Reasoning / 4th International Conference on Temporal Logic (TIME-ICTL 2003), 8-10 July 2003, Cairns, Queensland, Australia*, pages 155–165. IEEE Computer Society, 2003. [doi:10.1109/TIME.2003.1214891](https://doi.org/10.1109/TIME.2003.1214891).

- [14] Emanuel Kieronski. On the complexity of the two-variable guarded fragment with transitive guards. *Inf. Comput.*, 204(11):1663–1703, 2006. [doi:10.1016/j.ic.2006.08.001](https://doi.org/10.1016/j.ic.2006.08.001).
- [15] Orna Kupferman. Augmenting branching temporal logics with existential quantification over atomic propositions. *J. Log. Comput.*, 9(2):135–147, 1999. [doi:10.1093/logcom/9.2.135](https://doi.org/10.1093/logcom/9.2.135).
- [16] François Laroussinie and Nicolas Markey. Quantified CTL: expressiveness and complexity. *Logical Methods in Computer Science*, 10(4), 2014. [doi:10.2168/LMCS-10\(4:17\)2014](https://doi.org/10.2168/LMCS-10(4:17)2014).
- [17] François Laroussinie, Antoine Meyer, and Eudes Petonnet. Counting CTL. *Logical Methods in Computer Science*, 9(1), 2012. [doi:10.2168/LMCS-9\(1:3\)2013](https://doi.org/10.2168/LMCS-9(1:3)2013).
- [18] Kamal Lodaya and A. V. Sreejith. Two-variable first order logic with counting quantifiers: Complexity results. In *DLT*, volume 10396 of *Lecture Notes in Computer Science*, pages 260–271. Springer, 2017.
- [19] Anindya C. Patthak, Indrajit Bhattacharya, Anirban Dasgupta, Pallab Dasgupta, and P. P. Chakrabarti. Quantified computation tree logic. *Inf. Process. Lett.*, 82(3):123–129, 2002. [doi:10.1016/S0020-0190\(01\)00260-5](https://doi.org/10.1016/S0020-0190(01)00260-5).
- [20] Sylvain Schmitz. Complexity hierarchies beyond elementary. *TOCT*, 8(1):3:1–3:36, 2016. [doi:10.1145/2858784](https://doi.org/10.1145/2858784).
- [21] Aravinda Prasad Sistla. *Theoretical Issues in the Design and Verification of Distributed Systems*. PhD thesis, Cambridge, MA, USA, 1983. AAI8403047.
- [22] Peter van Emde Boas. The convenience of tilings. 1997.
- [23] Volker Weber. On the complexity of branching-time logics. In Erich Grädel and Reinhard Kahle, editors, *Computer Science Logic, 23rd international Workshop, CSL 2009, 18th Annual Conference of the EACSL, Coimbra, Portugal, September 7-11, 2009. Proceedings*, volume 5771 of *Lecture Notes in Computer Science*, pages 530–545. Springer, 2009. [doi:10.1007/978-3-642-04027-6_38](https://doi.org/10.1007/978-3-642-04027-6_38).

A Encoding tiles

This section will be dedicated to presenting formulae representing a given instance I of tiling problem. Then, we will prove that all presented formulae are correct i.e. they are satisfiable iff a system I has a solution. We stress here that all formulae presented below are written under the assumption that φ_{grid} holds. From now we keep in mind that $I = ((T, V, H), n, top_0, \dots, top_{n-1}, bot)$, i.e. an instance of a tiling problem, is given.

We first produce a formula stating that every cell has exactly one tile on it. For each tile t from T we introduce a fresh unary symbol t corresponding to such tile. It seems to be a bit ambiguous but will make clearly make our reasoning simpler.

$$\varphi_{tiles} \stackrel{\text{def}}{=} \exists T|_T \forall x \downarrow_x^{2n} \rightarrow @_x^{2n} \left(\bigvee_{t \in T} t \wedge \bigwedge_{t, t' \in T} (\neg t \vee \neg t') \right)$$

The correctness of the above formula is straightforward.

The next two formulae says that tiling respect V and H rules. They are nearly the same as formulae for enforcing an existence of H and V successors in grids.

$$\varphi_{hRules} \stackrel{\text{def}}{=} \forall x \downarrow_x^{2n} \wedge @_x^{2n} (H_n \neq 2^n - 1) \rightarrow \exists y_h \downarrow_{y_h}^{2n} \varphi_{succH}(x, y_h) \wedge \bigvee_{(t, t') \in H} (@_x^{2n}(t) \wedge @_{y_h}^{2n}(t'))$$

$$\varphi_{vRules} \stackrel{\text{def}}{=} \forall x \downarrow_x^{2n} \wedge @_x^{2n} (V_n \neq 2^n - 1) \rightarrow \exists y_v \downarrow_{y_v}^{2n} \wedge \varphi_{succV}(x, y_v) \wedge \bigvee_{(t, t') \in V} (@_x^{2n}(t) \wedge @_{y_v}^{2n}(t'))$$

The last thing to do, before claiming correctness of the reduction, is to ensure the proper tiling of the first row and that the tile *bot* appear somewhere at the last row.

The former one can be done in this way:

$$\varphi_{init} \stackrel{\text{def}}{=} \bigwedge_{i=0}^{n-1} \exists x_i \downarrow_{x_i}^{2n} \wedge @_{x_i}^{2n} (H_n = 0 \wedge V_n = i \wedge t_i)$$

The latter one can be done in that way:

$$\varphi_{bot} \stackrel{\text{def}}{=} \exists x_{fin} \downarrow_{x_{fin}}^{2n} \wedge @_{x_{fin}}^{2n} (H_n = 2^n - 1 \wedge bot)$$

Let φ_{tiling} be a conjunction of the previous formulae and let $\varphi_{reduction(I)}$ be $\varphi_{grid} \wedge \varphi_{tiling}$. We claim that our reduction is correct i.e. that the following theorem holds:

Theorem A.1. *A QCTL_{EX} formula $\varphi_{reduction(I)}$ is satisfiable over binary trees if and only if a tiling system I has a solution.*

Proof. Take an arbitrary instance I of a tiling problem. Assume that $\varphi_{reduction(I)}$ is satisfiable and let $\mathfrak{T}$ be its tree model. We claim that a system I has a solution.

We first going to prove that if $\mathfrak{T}$ satisfies φ_{grid} , then $\mathfrak{T}$ has exactly $2^n \times 2^n$ nodes at the level $2n$ and each of them is labeled by a different pair (h, v) of numbers written on H_n and V_n predicates. Observe that since $\mathfrak{T}$ satisfies $\varphi_{firstCell}$ then there is for sure a node at level $2n$ labeled with $H_n = 0$ and $V_n = 0$. Then, for arbitrary node at that level, using formulae $\varphi_{hSuccessors}$ and $\varphi_{vSuccessors}$ we know that each node has a proper H and V successor. Finally formula $\varphi_{uniqueness}$ ensure us that there are no node at level $2n$ with the same labeling with predicates H_n and

V_n . Hence, we have $2^n \times 2^n$ nodes, since it is the total number of different pairs of numbers represented on such predicates. Then, we turn our attention to the second part of $\varphi_{reduction(I)}$ formula i.e. a formula φ_{tiling} . Since a tree $\mathfrak{T}$ satisfies φ_{init} , we know that first n cells in a grid are labeled correctly. Additionally, we know that every cell respect φ_{hRules} and φ_{vRules} , which states that two successors do not violate H and V relations from an instance I . Because there all cells are filled with a tile (a formula φ_{tiles}) and they are not violating rules thus tiling is correct. We also know that a tile bot is somewhere in the last row (a formula φ_{bot}), which solves the last rule of tiling system. To get a solution for the tiling system I , we simply take a tiles in the way they are written in the tree i.e. cell at position (i, j) should be labeled with the same tile as the corresponding node of tree $\mathfrak{T}$ at level $2n$ labeled with $H_n=i$ and $V_n=j$. Thus an instance of a tiling system I has a solution.

The proof in the other direction is straightforward. First of all we build a binary tree $\mathfrak{T}$ in such a way that it has exactly $2^n \times 2^n$ nodes at the level $2n$ and we label each of these nodes with a predicates H_n and V_n in such a way that none of them carries the same pair (h, v) of numbers represented on such predicates. Such tree can be easily constructed and it satisfies φ_{grid} formula. Then, we simply take a solution of a tiling system I and for each tile laying on cell at position (i, j) we put the very same tile on node at level $2n$ labeled with $H_n=i$ and $V_n=j$. Since we used tiling from the solution of I it surely respect φ_{tiling} . Thus we have build a model for $\varphi_{reduction(I)}$, which finishes the proof. □